

MiniMarket — Sistema de gestion de inventario y ventas

Proyecto Final — Programacion III

Estudiante	Carlos Vidal
Matricula	[MATRICULA]
Asignatura	Programacion III
Docente	Kelyn Tejada
Proyecto de software	MiniMarket (rd-market)
Fecha de entrega	14 de agosto de 2026

Indice

1. Introduccion
2. Estrategia de trabajo (planificacion)
 1. Nombre del proyecto de software
 2. Tecnologia para aplicar
 3. Objetivo del proyecto
 4. Alcance del proyecto
 5. Cronograma del proyecto
 6. Definicion del primer Release
3. Metodologia Scrum
 1. Tareas a ejecutar
 2. Equipo de trabajo
 3. Herramientas
 4. Epicas
 5. Ceremonias de Scrum
 6. Historias de usuario
4. Plan de pruebas
 1. Requerimientos funcionales y no funcionales
 2. Criterios de aceptacion y rechazo de pruebas
 3. Herramientas de pruebas
 4. Cronograma de ejecucion de pruebas
 5. Plantilla para casos de prueba
 6. Equipos de pruebas y responsables
 7. Plan de automatizacion de pruebas
 8. Ejecucion y demostracion
5. Demostracion y entregables
6. Conclusiones
7. Bibliografia

1. Introduccion

Este documento recoge la planificacion, la gestion agil y el plan de pruebas del proyecto final de Programacion III. El sistema desarrollado es **MiniMarket**, una aplicacion web de gestion de inventario y ventas para un comercio pequeno, construida sobre Next.js con una base de datos relacional.

El trabajo parte de una entrega anterior en la que ya existian el CRUD de productos, usuarios y compras y una suite de once pruebas automatizadas con Selenium. El incremento que se presenta aqui sustituye la autentificacion improvisada de aquella version por **Better Auth**, una libreria de autentificacion para TypeScript, e introduce **control de acceso por roles** con su plugin de administracion. Con ello el sistema pasa de tener un login meramente decorativo a tener cuentas reales, contraseñas cifradas, sesiones revocables y permisos diferenciados entre la administracion y la clientela.

El documento se organiza en tres bloques: la planificacion del proyecto, la aplicacion de la metodologia Scrum y el plan de pruebas. Cierra con las conclusiones y los enlaces a los entregables.

2. Estrategia de trabajo (planificacion)

2.1 Nombre del proyecto de software

MiniMarket — Sistema de gestion de inventario y ventas. El repositorio se llama `rd-market`.

El nombre describe el ambito del sistema: un comercio de barrio que necesita llevar el control de su catalogo, de las personas registradas y de las ventas realizadas, sin la complejidad de un ERP.

2.2 Tecnologia para aplicar

Capa	Herramienta	Version	Por que
Lenguaje	TypeScript	6.0	Tipado estatico de extremo a extremo; los errores de datos aparecen al compilar y no en produccion
Framework	Next.js (App Router)	16.2	Un solo proyecto resuelve interfaz, rutas de API y logica de servidor mediante <i>server components</i> y <i>server actions</i>
Interfaz	React + Tailwind CSS	19.2 / 4.3	React viene incluido en Next; Tailwind evita mantener hojas de estilo aparte
ORM	Prisma	7.9	Esquema declarativo, migraciones versionadas y cliente tipado generado a partir del modelo
Base de datos	SQLite (via libSQL)	—	Archivo unico, sin servidor que instalar: adecuada para un proyecto academico y suficiente para el volumen del caso
Autenticacion	Better Auth + plugin <code>admin</code>	1.6	Cifrado de contraseñas, sesiones en base de datos y gestion de roles sin escribir criptografia propia
Pruebas E2E	Selenium WebDriver	4.46	Exigido por la asignatura; controla un navegador real, de modo que se prueba lo mismo que ve la persona usuaria
Ejecutor de pruebas	Mocha + Chai	11.8 / 6.2	Sintaxis clara de suites y aserciones, con soporte nativo de <code>async/await</code>
Reportes	mochawesome	8.0	Genera el informe HTML con capturas incrustadas que pide la rubrica
Control de versiones	Git + GitHub	—	Historial por ramas y revision mediante Pull Requests
Gestion agil	Jira Cloud	—	Epicas, historias de usuario, puntos de historia y tablero publico

2.3 Objetivo del proyecto

Desarrollar una aplicacion web que permita a un minimarket **administrar su inventario y registrar sus ventas**, garantizando que cada persona acceda unicamente a la informacion y a las operaciones que le corresponden segun su rol, y demostrando la calidad del resultado mediante una bateria de pruebas automatizadas.

Objetivos especificos:

1. Ofrecer un CRUD completo de productos, usuarios y compras sobre una base de datos relacional.
2. Mantener la coherencia del inventario: registrar una compra descuenta stock y anularla lo devuelve, siempre dentro de una transaccion.
3. Autenticar a las personas usuarias con email y contraseña, guardando la contraseña cifrada.

4. Diferenciar dos roles —**administrador** y **cliente**— y aplicar sus permisos tanto en la interfaz como en la API.
5. Automatizar con Selenium al menos un caso de prueba por historia de usuario, con reporte HTML y capturas de pantalla.
6. Trabajar con un flujo Git de ramas y Pull Requests, y con un tablero Jira publico.

2.4 Alcance del proyecto

Dentro del alcance

Modulo	Contenido
Autenticacion	Registro publico, inicio y cierre de sesion con email y contrasena; sesion en cookie <code>httpOnly</code> respaldada por base de datos
Autorizacion	Roles <code>admin</code> y <code>user</code> ; proteccion de paginas, <i>server actions</i> y API REST
Productos	Alta, consulta, edicion y baja; consulta abierta a cualquier sesion, modificacion reservada a la administracion
Usuarios	Alta, edicion, cambio de rol, bloqueo y baja; seccion exclusiva de la administracion
Compras	Registro y anulacion con ajuste transaccional del stock; cada cliente ve solo las suyas
API REST	Seis endpoints bajo <code>/api</code> , todos autenticados y autorizados por rol
Pruebas	16 casos automatizados con Selenium sobre navegador Chrome, con reporte HTML y capturas

Fuera del alcance

- Proveedores externos de identidad (Google, GitHub), verificacion por correo y recuperacion de contrasena. Better Auth los soporta, pero quedan fuera de esta entrega.
- Pasarela de pago, facturacion fiscal y control de proveedores.
- Aplicacion movil nativa y funcionamiento sin conexion.
- Despliegue en produccion con base de datos gestionada: el sistema corre en local sobre SQLite.
- Internacionalizacion: la interfaz esta solo en espanol.

2.5 Cronograma del proyecto

El proyecto se desarrollo en cuatro sprints de una semana. La columna *Responsable* refleja el rol que se asume en cada actividad (ver [seccion 3.2](#)).

#	Actividad	Sprint	Fechas	Responsable	Estado
1	Análisis de requisitos y definición del alcance	Sprint 1	28-29 jul 2026	Product Owner	Completada
2	Configuración del proyecto base (Next.js, Tailwind, Prisma, SQLite)	Sprint 1	30 jul 2026	Desarrollo	Completada
3	Modelado de datos y migraciones	Sprint 1	30 jul 2026	Desarrollo	Completada
4	CRUD de productos (páginas y API)	Sprint 1	30 jul 2026	Desarrollo	Completada
5	CRUD de usuarios (páginas y API)	Sprint 1	30 jul 2026	Desarrollo	Completada
6	Módulo de compras con descuento transaccional de stock	Sprint 2	30-31 jul 2026	Desarrollo	Completada
7	Navegación, estilos compartidos y validaciones	Sprint 2	31 jul 2026	Desarrollo	Completada
8	Redacción de las historias HU-01 a HU-05 y publicación en Jira	Sprint 3	4 ago 2026	Product Owner	Completada
9	Infraestructura de pruebas (Selenium, Mocha, mochawesome)	Sprint 3	4 ago 2026	QA	Completada
10	Automatización de los once primeros casos (CP-01 a CP-11)	Sprint 3	4-5 ago 2026	QA	Completada
11	Tablero público de Jira y reporte HTML de la primera tanda	Sprint 3	5 ago 2026	Scrum Master	Completada
12	Migración de la autenticación a Better Auth	Sprint 4	14 ago 2026	Desarrollo	Completada
13	Roles, administración de usuarios y protección de la API	Sprint 4	14 ago 2026	Desarrollo	Completada
14	Historias HU-06 a HU-10 y épica E-02 en Jira	Sprint 4	14 ago 2026	Product Owner	Completada
15	Automatización de los casos CP-12 a CP-16	Sprint 4	14 ago 2026	QA	Completada
16	Regresión completa, reporte final y documentación	Sprint 4	14 ago 2026	QA / Scrum Master	Completada
17	Video demostrativo y entrega	Sprint 4	14 ago 2026	Scrum Master	Completada

Resumen por sprint

Sprint	Fechas	Objetivo	Puntos comprometidos
Sprint 1	28 jul – 30 jul 2026	Proyecto base y CRUD de productos y usuarios	13
Sprint 2	30 jul – 2 ago 2026	Compras, navegación y validaciones	8
Sprint 3	3 ago – 5 ago 2026	Historias HU-01 a HU-05 y automatización inicial	21
Sprint 4	8 ago – 14 ago 2026	Better Auth, roles y ampliación de la suite	26

2.6 Definición del primer Release

El **Release 1.0** entrega un minimarket operativo: una persona administradora mantiene el catálogo y las cuentas, y la clientela consulta el catálogo y su propio historial de compras. La novedad principal de esta versión frente al prototipo anterior es que **la autenticación y los permisos son reales**: las contraseñas se guardan cifradas, las sesiones viven en la base de datos y ninguna ruta —ni de la interfaz ni de la API— queda accesible sin las credenciales adecuadas.

Que podrá hacer el sistema en su primera versión

Cualquier visitante

- Crear una cuenta desde `/registro` indicando nombre, correo y contraseña. La cuenta nace con el rol de **cliente**.
- Iniciar sesión en `/login`. Sin sesión, toda ruta protegida redirige al formulario de acceso.

Persona con rol cliente

- Consultar el catálogo completo de productos con su categoría, precio y stock.
- Registrar compras a su propio nombre, con validación de stock disponible.
- Consultar y anular únicamente **sus** compras.
- Cerrar sesión, lo que revoca la sesión en el servidor.

Persona con rol administrador

- Todo lo anterior, y además:
- Crear, editar y eliminar productos del catálogo.
- Administrar cuentas: alta, edición, cambio de rol, bloqueo y desbloqueo, y baja.
- Consultar las compras de toda la clientela, con la columna del comprador, y registrar compras a nombre de terceros.

Requerimientos funcionales

ID	Requerimiento	Historia	Prioridad
RF-01	El sistema permite crear una cuenta con nombre, email y contraseña, asignando el rol <code>user</code> por defecto	HU-06	Alta
RF-02	El sistema autentica con email y contraseña mediante Better Auth y crea una sesion persistente	HU-01, HU-07	Alta
RF-03	El sistema cierra la sesion revocandola en el servidor y borrando la cookie	HU-07	Alta
RF-04	El sistema distingue los roles <code>admin</code> y <code>user</code> y aplica sus permisos en paginas, <i>server actions</i> y API	HU-08, HU-09, HU-10	Alta
RF-05	El administrador da de alta, edita, cambia de rol, bloquea y elimina cuentas	HU-08	Alta
RF-06	El administrador crea, edita y elimina productos del catalogo	HU-02, HU-04, HU-05, HU-10	Alta
RF-07	Cualquier sesion consulta el catalogo con nombre, categoria, precio y stock	HU-03	Alta
RF-08	El sistema registra compras descontando el stock del producto dentro de una transaccion	HU-09	Alta
RF-09	La anulacion de una compra devuelve las unidades al stock, tambien de forma transaccional	HU-09	Media
RF-10	Cada cliente ve unicamente sus compras; el administrador ve las de todos	HU-09	Alta
RF-11	El sistema valida los datos de entrada y devuelve mensajes en espanol junto al formulario	HU-02, HU-06	Media
RF-12	El sistema expone una API REST de productos, usuarios y compras, autenticada y autorizada por rol	HU-08, HU-10	Media

Requerimientos no funcionales

ID	Requerimiento	Como se cumple	Como se verifica
RNF-01	Seguridad de credenciales: ninguna contraseña se guarda ni se muestra en claro	Better Auth cifra con <code>scrypt</code> y guarda el hash en la tabla <code>account</code>	Inspeccion de la base de datos; el formulario de edicion no precarga la contraseña
RNF-02	Sesiones revocables: la sesion debe poder invalidarse desde el servidor	Las sesiones viven en la tabla <code>sesion</code> ; bloquear una cuenta revoca las suyas	Bloqueo desde <code>/usuarios</code> y reintento de acceso
RNF-03	Cookie protegida: la cookie de sesion no es accesible desde JavaScript	Cookie <code>httpOnly</code> , <code>sameSite=lax</code> , caducidad de 7 dias, <code>secure</code> en produccion	Inspeccion de cabeceras en la respuesta de <code>sign-in</code>
RNF-04	Defensa en profundidad: la autorizacion no depende de ocultar botones	El rol se comprueba en la pagina, en la <code>server action</code> y en el route handler	CP-14 y CP-16; pruebas manuales sobre la API con <code>curl</code>
RNF-05	Integridad de datos: el stock nunca queda descuadrado	Compra y ajuste de stock ocurren en una sola transaccion de Prisma	Revision del listado de productos tras registrar y anular
RNF-06	Trazabilidad: cada cambio debe poder rastrearse	Ramas <code>feature/*</code> , Pull Requests con descripcion y merge commits	Historial de <code>git log --graph</code>
RNF-07	Mantenibilidad: el codigo debe estar tipado y sin avisos	<code>tsc --noEmit</code> y ESLint sin errores	Ejecucion en cada entrega
RNF-08	Usabilidad: los mensajes de error deben ser comprensibles y aparecer junto al formulario	Mensajes en espanol renderizados por el componente <code>MensajeError</code>	CP-02, CP-05, CP-13
RNF-09	Reproducibilidad: cualquiera debe poder levantar el proyecto	<code>npm install</code> , migraciones versionadas y <code>db:seed</code> con datos de ejemplo	Instalacion desde un clon limpio
RNF-10	Rendimiento: la suite completa de pruebas no debe exceder los 3 minutos	16 casos en modo headless	Tiempo registrado en el reporte de mochawesome

3. Metodologia Scrum

3.1 Tareas a ejecutar

El trabajo se descompuso en tareas asignables a un sprint y verificables por separado.

Sprint 1 — Base del sistema

Tarea	Descripcion	Historia
T-01	Inicializar el repositorio y el flujo de ramas <code>main</code> , <code>development</code> y <code>qa</code>	—
T-02	Configurar Next.js con TypeScript y Tailwind	—
T-03	Configurar Prisma sobre SQLite y crear la primera migracion	—
T-04	Modelar <code>Producto</code> y su capa de acceso a datos	HU-02
T-05	Construir las paginas de listado, alta y edicion de productos	HU-02, HU-03, HU-04
T-06	Publicar los endpoints REST de productos	—
T-07	Modelar <code>Usuario</code> y construir su CRUD	HU-08

Sprint 2 — Compras y consolidacion

Tarea	Descripcion	Historia
T-08	Modelar <code>Compra</code> con sus relaciones y borrado en cascada	HU-09
T-09	Registrar compras descontando stock dentro de una transaccion	HU-09
T-10	Construir el historial de compras y la anulacion	HU-09
T-11	Anadir la barra de navegacion y centralizar los estilos	—
T-12	Validar los datos en la capa de negocio y mostrar los mensajes en el formulario	HU-02

Sprint 3 — Historias y automatizacion

Tarea	Descripcion	Historia
T-13	Redactar HU-01 a HU-05 con criterios de aceptacion y rechazo	HU-01...HU-05
T-14	Crear el espacio de Jira y publicar la epica E-01 con sus historias	—
T-15	Montar Selenium, Mocha y mochawesome, con capturas automaticas	—
T-16	Escribir los <i>page objects</i> de login, listado y formulario	—
T-17	Automatizar CP-01 a CP-11	HU-01...HU-05
T-18	Abrir el tablero de Jira al publico y generar el reporte HTML	—

Sprint 4 — Autenticacion, roles y ampliacion

Tarea	Descripcion	Historia
T-19	Generar el esquema de Better Auth y migrar la base de datos	HU-07
T-20	Configurar la instancia de Better Auth con el plugin <code>admin</code>	HU-07, HU-08
T-21	Reescribir sesion, capa de acceso y proxy sobre Better Auth	HU-07
T-22	Construir el registro publico en <code>/registro</code>	HU-06
T-23	Rehacer el <code>seed</code> para que las contraseñas queden cifradas	HU-07
T-24	Proteger los seis route handlers de la API con sesion y rol	HU-08, HU-10
T-25	Restringir la administracion de usuarios y el catalogo al rol <code>admin</code>	HU-08, HU-10
T-26	Filtrar las compras por usuario de la sesion	HU-09
T-27	Redactar HU-06 a HU-10 y publicar la epica E-02 en Jira	HU-06...HU-10
T-28	Automatizar CP-12 a CP-16 y estabilizar la suite	HU-06, HU-08, HU-09, HU-10
T-29	Ejecutar la regresion completa y generar el reporte final	—
T-30	Redactar la documentacion y grabar el video demostrativo	—

3.2 Equipo de trabajo

El proyecto es de caracter **individual**, de modo que el equipo lo forma una sola persona. Esto es viable porque Next.js es un *framework fullstack*: interfaz, logica de servidor, rutas de API y acceso a datos viven en un unico proyecto y en un unico lenguaje, sin necesidad de coordinar equipos separados de frontend y backend.

Integrante	Rol Scrum	Responsabilidades	Habilidades requeridas
Carlos Vidal	Product Owner	Definir el alcance, redactar las historias con sus criterios, priorizar el backlog y aceptar el incremento	Análisis de requisitos, redacción de historias, criterio de negocio
Carlos Vidal	Scrum Master	Planificar los sprints, mantener el tablero de Jira al día, moderar las ceremonias y retirar impedimentos	Scrum, gestión de Jira, organización del trabajo
Carlos Vidal	Development Team	Diseñar e implementar el sistema, modelar los datos y revisar el código en cada Pull Request	TypeScript, React, Next.js, Prisma, SQL, Git
Carlos Vidal	QA	Diseñar el plan de pruebas, automatizar los casos, ejecutar la regresión y reportar los defectos	Selenium WebDriver, Mocha, Chai, diseño de casos de prueba

Como se repartiría en un proyecto real. Aunque aquí una sola persona asume los cuatro roles, la separación no es un formalismo: cada rol se ejerce en un momento distinto y con una mentalidad distinta. En un equipo real la asignación habitual sería:

Rol	Personas	Dedicación	Que aportaría
Product Owner	1	Parcial	Unico responsable del backlog; interlocutor con el cliente
Scrum Master	1	Parcial	Facilita las ceremonias y protege al equipo de interrupciones
Development Team	3 a 5	Completa	Perfiles mixtos de frontend, backend y base de datos, autoorganizados
QA	1 a 2	Completa	Integrado en el equipo de desarrollo, no como fase posterior

La disciplina que si se ha respetado, y que es la que hace posible sostener los cuatro roles a la vez, es la del **flujo de trabajo**: ninguna tarea entra en `development` sin pasar por una rama `feature/*` y un Pull

Request revisado. La revision obliga a mirar el propio codigo con la mentalidad de quien no lo escribio, que es justamente lo que aporta un segundo par de ojos en un equipo real.

3.3 Herramientas

Herramienta	Uso en el proyecto
Jira Cloud	Backlog, epicas, historias con criterios y puntos, y tablero publico de seguimiento
GitHub	Repositorio, Pull Requests y revision de codigo
Git	Control de versiones con ramas permanentes <code>main</code> , <code>qa</code> y <code>development</code> , y ramas <code>feature/*</code>
Visual Studio Code	Entorno de desarrollo, con ESLint y el servidor de TypeScript
Prisma Studio	Inspeccion visual de la base de datos durante el desarrollo
Chrome + ChromeDriver	Navegador sobre el que se ejecutan las pruebas de Selenium
mochawesome	Reporte HTML de las pruebas con las capturas incrustadas
npm	Gestor de dependencias y ejecutor de los scripts del proyecto

Flujo de trabajo entre Jira y GitHub. Cada historia de Jira se implementa en una rama `feature/*` creada desde `development`. Al terminarla se abre un Pull Request que describe el cambio y se integra con `merge commit` —nunca con `squash`— para que el historial conserve la traza de cada rama. Cuando el sprint cierra, `development` se promociona a `qa` y, tras la regresion, `qa` se promociona a `main`. Ninguna rama permanente recibe commits directos.

3.4 Epicas

Las diez historias se agrupan en dos epicas que se corresponden con los dos grandes bloques funcionales del sistema.

Epica	Titulo	Historias	Puntos	Sprints
E-01	Automatizacion CRUD con Selenium	HU-01 a HU-05	21	1 a 3
E-02	Autenticacion y control de acceso con Better Auth	HU-06 a HU-10	26	4

E-01 — Automatizacion CRUD con Selenium. Agrupa el nucleo de gestion del inventario: entrar a la aplicacion y mantener el catalogo de productos, junto con la suite de pruebas que lo respalda. Es la base sobre la que se apoya todo lo demas.

E-02 — Autenticacion y control de acceso con Better Auth. Agrupa las historias que convierten el acceso en algo real y gobernado por permisos: registro publico, sesiones gestionadas por Better Auth, administracion de cuentas y roles, privacidad de las compras y catalogo de solo lectura para la clientela.

3.5 Ceremonias de Scrum

Sprints de una semana, de lunes a viernes. Al tratarse de un equipo de una persona, la *daily* se resuelve como una revision escrita del tablero al comenzar la jornada, y la *review* como una demostracion grabada del incremento.

Ceremonia	Frecuencia	Duracion	Momento
Sprint Planning	Al inicio de cada sprint	1 h	Lunes 09:00
Daily Stand-up	Diaria	10 min	De lunes a viernes, 09:00
Sprint Review	Al cierre de cada sprint	45 min	Viernes 16:00
Sprint Retrospective	Al cierre de cada sprint	30 min	Viernes 17:00
Refinamiento del backlog	Semanal	45 min	Miercoles 15:00

Calendario efectivo

Sprint	Planning	Dailies	Review	Retrospective
Sprint 1	Lun 28 jul 2026, 09:00	28 jul – 31 jul, 09:00	Vie 31 jul 2026, 16:00	Vie 31 jul 2026, 17:00
Sprint 2	Lun 3 ago 2026, 09:00	3 ago – 7 ago, 09:00	Vie 7 ago 2026, 16:00	Vie 7 ago 2026, 17:00
Sprint 3	Lun 3 ago 2026, 09:00	3 ago – 5 ago, 09:00	Mie 5 ago 2026, 16:00	Mie 5 ago 2026, 17:00
Sprint 4	Lun 10 ago 2026, 09:00	10 ago – 14 ago, 09:00	Vie 14 ago 2026, 16:00	Vie 14 ago 2026, 17:00

Resultado de las retrospectivas

Sprint	Que funciona	Que mejorar	Accion tomada
1	El esquema de Prisma estabilizo el modelo desde el primer día	Las validaciones estaban repartidas entre paginas y API	Se centralizaron en la capa <code>src/lib</code>
2	La transaccion de compra evito descuadres de stock	La interfaz repetia clases de Tailwind	Se creo <code>src/components/ui.ts</code>
3	Los <i>page objects</i> dejaron los casos muy legibles	Los selectores dependian de textos literales	Se documentaron en las historias como criterios
4	Better Auth elimino la criptografia hecha a mano	Dos casos fallaban de forma intermitente por la hidratacion de React	Se anadieron reintentos en el soporte de pruebas

3.6 Historias de usuario

Las diez historias, con sus criterios de aceptacion y rechazo completos, estan en `docs/historias-usuario.md` y publicadas en el tablero de Jira. Resumen:

ID	Historia	Epica	Puntos	Prioridad	Casos
HU-01	Iniciar sesion en MiniMarket	E-01	5	Alta	CP-01, CP-02, CP-03
HU-02	Crear un producto	E-01	5	Alta	CP-04, CP-05, CP-06
HU-03	Consultar el listado de productos	E-01	3	Alta	CP-07, CP-08
HU-04	Editar un producto existente	E-01	5	Alta	CP-09, CP-10
HU-05	Eliminar un producto	E-01	3	Media	CP-11
HU-06	Registrarme con email y contrasena	E-02	3	Alta	CP-12, CP-13
HU-07	Iniciar y cerrar sesion con Better Auth	E-02	5	Alta	CP-01, CP-02, CP-03
HU-08	Administrar usuarios y sus roles	E-02	8	Alta	CP-14
HU-09	Ver unicamente mis compras	E-02	5	Alta	CP-15
HU-10	Solo la administracion modifica el catalogo	E-02	5	Alta	CP-16
			47		16 casos

Estimacion. Se uso la sucesion de Fibonacci (1, 2, 3, 5, 8) sobre una escala relativa en la que 3 puntos equivalen a una pantalla con su formulario y sus validaciones. HU-08 recibe 8 puntos por ser la historia que mas superficie toca: pantalla, seis operaciones distintas de la API del plugin `admin`, y reglas para que la administracion no pueda bloquearse a si misma. HU-06 recibe solo 3 porque reutiliza el patron de formulario y de mensajes de error ya existente.

4. Plan de pruebas

4.1 Requerimientos funcionales y no funcionales

Los requerimientos verificados por las pruebas son los definidos en la [seccion 2.6](#). Su cobertura es la siguiente:

Requerimiento	Historia	Casos que lo cubren	Tipo de verificacion
RF-01 Registro de cuenta	HU-06	CP-12, CP-13	Automatizada
RF-02 Autenticacion	HU-01, HU-07	CP-01, CP-02, CP-03	Automatizada
RF-03 Cierre de sesion	HU-07	CP-01 (comprueba el control), manual	Mixta
RF-04 Roles y permisos	HU-08, HU-09, HU-10	CP-14, CP-15, CP-16	Automatizada
RF-05 Administracion de cuentas	HU-08	CP-14 y pruebas manuales de alta, rol y bloqueo	Mixta
RF-06 Mantenimiento del catalogo	HU-02, HU-04, HU-05	CP-04 a CP-06, CP-09 a CP-11	Automatizada
RF-07 Consulta del catalogo	HU-03	CP-07, CP-08	Automatizada
RF-08 Registro de compra con descuento de stock	HU-09	Manual, apoyada en CP-15	Manual
RF-09 Anulacion con devolucion de stock	HU-09	Manual	Manual
RF-10 Privacidad de las compras	HU-09	CP-15	Automatizada
RF-11 Validacion y mensajes	HU-02, HU-06	CP-05, CP-10, CP-13	Automatizada
RF-12 API autenticada y autorizada	HU-08, HU-10	Manual con <code>curl</code> sobre los seis endpoints	Manual
RNF-01 Contraseñas cifradas	—	Inspeccion de la tabla <code>account</code>	Manual
RNF-04 Defensa en profundidad	—	CP-14, CP-16 mas comprobacion de la API	Mixta
RNF-05 Integridad del stock	—	Comparacion del stock antes y despues	Manual
RNF-10 Duracion de la suite	—	Tiempo del reporte de mochawesome	Automatizada

4.2 Criterios de aceptacion y rechazo de pruebas

Criterios de aceptacion de un caso de prueba. Un caso se considera **superado** cuando:

1. Todas sus aserciones se cumplen, sin excepciones ni tiempos de espera agotados.
2. La aplicacion queda en el estado esperado, comprobado desde la interfaz y no solo desde la base de datos.
3. Los mensajes que se muestran coinciden **literalmente** con los definidos en los criterios de la historia.
4. La captura de pantalla adjunta al reporte muestra el estado final descrito.
5. El caso se ejecuta de forma independiente: no depende del orden ni del resultado de otro caso.

Criterios de rechazo. Un caso se considera **fallido** cuando ocurre cualquiera de estas situaciones:

1. Una asercion no se cumple o el elemento esperado no aparece dentro del tiempo de espera.
2. La aplicacion responde con un error de servidor (500) o con una excepcion no controlada.
3. El mensaje de error mostrado no coincide con el especificado, aunque su sentido sea equivalente.
4. Una operacion reservada a un rol resulta accesible para otro, por la interfaz o por la API.
5. El caso solo pasa cuando se ejecuta en un orden concreto, lo que indica dependencia oculta entre casos.

Criterios de salida del ciclo de pruebas. La entrega se acepta cuando:

- El **100 %** de los casos automatizados pasa (16 de 16).
- Cada historia de usuario tiene **al menos un** caso automatizado asociado.
- No queda ningun defecto abierto de severidad **critica** o **alta**.
- `npx tsc --noEmit` y `npm run lint` terminan sin errores.
- El reporte HTML y las capturas estan generados y accesibles.

Clasificacion de defectos

Severidad	Definicion	Plazo de correccion
Critica	Impide usar el sistema o expone datos de otras personas	Inmediato; bloquea la entrega
Alta	Una funcion principal no opera como se especifico	Dentro del mismo sprint; bloquea la entrega
Media	Funcion secundaria afectada, con alternativa disponible	Sprint siguiente
Baja	Detalle cosmetico o de redaccion	Backlog

4.3 Herramientas de pruebas

Herramienta	Version	Funcion	Justificacion
Selenium WebDriver	4.46	Controla el navegador	Lo exige la asignatura y es el estandar de la industria para pruebas E2E. Conduce un Chrome real por el protocolo WebDriver, de modo que se ejercita exactamente lo que ve la persona usuaria: el HTML renderizado, la validacion nativa del navegador y las cookies. Ademas es independiente del framework, asi que la suite sobreviviria a un cambio de Next.js por otra tecnologia
Mocha	11.8	Organiza y ejecuta las suites	Soporte nativo de <code>async/await</code> , imprescindible cuando cada instruccion de Selenium es una promesa. Sus <i>hooks</i> globales (<code>mochaHooks</code>) permiten preparar la base de datos una sola vez y abrir un navegador limpio por caso, sin duplicar codigo
Chai	6.2	Aserciones	Sintaxis <code>expect(...).to.equal(...)</code> , que se lee casi como el criterio de aceptacion redactado en la historia
mocha-wesome	8.0	Reporte HTML y JSON	Genera el informe con graficos y tiempos que pide la rubrica, y permite adjuntar capturas a cada caso con <code>addContext</code> . La opcion <code>inlineAssets</code> produce un HTML autocontenido, que se puede abrir sin servidor
ChromeDriver / Chrome	141	Navegador bajo prueba	Es el navegador mas usado por la clientela objetivo. El modo <code>--headless=new</code> permite ejecutar la suite sin escritorio
Prisma	7.9	Preparacion de datos	El script <code>db:seed</code> deja la base en un estado conocido antes de cada corrida
curl	—	Pruebas manuales de la API	Verifica los codigos 401 y 403 de los endpoints sin pasar por la interfaz

Por que no otras herramientas. Se valoraron Cypress y Playwright, mas comodas para pruebas E2E modernas, pero la asignatura exige explicitamente Selenium. Para las pruebas unitarias habria encajado Vitest; se descarto porque la logica de negocio del proyecto es fina y el valor de las pruebas esta en los flujos completos, no en funciones aisladas. Se descarto **Selenium IDE** por estar expresamente prohibido en el enunciado.

4.4 Cronograma de ejecucion de pruebas

Fase	Actividad	Tipo	Fechas	Responsable
1	Diseno del plan de pruebas y de los casos CP-01 a CP-11	Diseno	3-4 ago 2026	QA
2	Montaje del entorno (Selenium, Mocha, mochawesome, capturas)	Preparacion	4 ago 2026	QA
3	Pruebas manuales exploratorias del CRUD	Manual	4 ago 2026	QA
4	Automatizacion de CP-01 a CP-11	Automatiza-da	4-5 ago 2026	QA
5	Primera regresion completa y reporte	Automatiza-da	5 ago 2026	QA
6	Diseno de los casos CP-12 a CP-16 para las historias nuevas	Diseno	14 ago 2026	QA
7	Pruebas manuales de la API por rol con <code>curl</code>	Manual	14 ago 2026	QA
8	Adaptacion de la suite existente a Better Auth	Automatiza-da	14 ago 2026	QA
9	Automatizacion de CP-12 a CP-16	Automatiza-da	14 ago 2026	QA
10	Regresion final de los 16 casos y reporte definitivo	Automatiza-da	14 ago 2026	QA
11	Pruebas manuales de administracion de usuarios y de stock	Manual	14 ago 2026	QA
12	Revision del reporte y cierre del ciclo	Cierre	14 ago 2026	Scrum Master

Politica de ejecucion

Momento	Que se ejecuta	Modo
Antes de abrir un Pull Request	Suite completa (16 casos)	<code>npm test:e2e:headless</code> run
Al terminar una historia	Los casos de esa historia mas los de las historias que toca	Con navegador visible
Antes de promocionar a <code>qa</code>	Suite completa mas las pruebas manuales de la API	Mixto
Antes de promocionar a <code>main</code>	Regresion completa y revision del reporte	Mixto

4.5 Plantilla para casos de prueba

Toda la suite se documenta con la siguiente plantilla. Los campos de identificacion permiten rastrear el caso hasta la historia y hasta el requerimiento del que nace.

Campo	Contenido
ID del caso	Identificador unico, CP–nn
Historia de usuario	Historia que verifica, HU–nn
Requerimiento	Requerimiento funcional o no funcional asociado
Titulo	Que comprueba el caso, en una frase
Tipo	Camino feliz / Negativa / Limites
Prioridad	Alta / Media / Baja
Precondiciones	Estado que debe tener el sistema antes de empezar
Datos de prueba	Valores concretos que se introducen
Pasos	Secuencia numerada de acciones
Resultado esperado	Comportamiento que debe observarse
Resultado obtenido	Lo que ocurrio realmente en la ejecucion
Estado	Superado / Fallido / Bloqueado
Evidencia	Captura de pantalla adjunta al reporte
Automatizado	Si / No, y archivo que lo implementa
Responsable	Quien lo diseno y lo ejecuto

Ejemplo cumplimentado — CP-15

Campo	Contenido
ID del caso	CP-15
Historia de usuario	HU-09 Ver unicamente mis compras
Requerimiento	RF-10 Privacidad de las compras
Titulo	El cliente solo ve sus compras y la administradora las ve todas
Tipo	Camino feliz
Prioridad	Alta
Precondiciones	Base de datos con el <i>seed</i> cargado: una compra de Ana Perez (administradora) y una de Carlos Vidal (cliente)
Datos de prueba	carlos@minimarket.com / carlos1234 ; ana@minimarket.com / ana1234
Pasos	1. Iniciar sesion como carlos@minimarket.com . 2. Abrir /compras . 3. Contar las filas y leer su contenido. 4. Descartar la sesion. 5. Iniciar sesion como ana@minimarket.com . 6. Abrir /compras y volver a contar
Resultado esperado	El cliente ve el titulo "Mis compras" y una unica fila (Pan de molde), sin rastro de Leche entera 1L ni del correo de la otra cuenta. La administradora ve dos filas, la columna Usuario y ambos productos
Resultado obtenido	Coincide con lo esperado
Estado	Superado
Evidencia	tests-selenium/reportes/capturas/cp-15-feliz-el-cliente-solo-ve-sus-compras-y-la-administradora-las-ve-todas.png
Automatizado	Si — tests-selenium/08-mis-compras.spec.js
Responsable	Carlos Vidal (QA)

Ejemplo cumplimentado — CP-05

Campo	Contenido
ID del caso	CP-05
Historia de usuario	HU-02 Crear un producto
Requerimiento	RF-11 Validacion y mensajes
Título	Se rechaza un nombre formado solo por espacios
Tipo	Negativa
Prioridad	Alta
Precondiciones	Sesion iniciada con rol administrador, en <code>/productos/nuevo</code>
Datos de prueba	Nombre " ", precio 1, stock 1
Pasos	1. Escribir tres espacios en Nombre. 2. Rellenar precio y stock. 3. Pulsar Crear producto
Resultado esperado	El valor supera el <code>required</code> del navegador pero lo rechaza el servidor: se vuelve a <code>/productos/nuevo?error=...</code> con el mensaje "El nombre del producto es obligatorio." y el producto no se crea
Resultado obtenido	Coincide con lo esperado
Estado	Superado
Evidencia	<code>tests-selenium/reportes/capturas/cp-05-negativa-rechaza-un-nombre-que-solo-tiene-espacios.png</code>
Automatizado	Si — <code>tests-selenium/02-crear-producto.spec.js</code>
Responsable	Carlos Vidal (QA)

4.6 Equipos de pruebas y responsables

Al ser un proyecto individual, Carlos Vidal ejecuta todas las pruebas asumiendo el rol de QA. La tabla refleja quien ejecuta cada bloque y como se organizaria en un equipo real.

Bloque	Casos	Responsable	En un equipo real
Autenticacion y registro	CP-01, CP-02, CP-03, CP-12, CP-13	Carlos Vidal (QA)	QA funcional, con revision de un perfil de seguridad
CRUD de productos	CP-04 a CP-11	Carlos Vidal (QA)	QA funcional
Roles y permisos	CP-14, CP-16	Carlos Vidal (QA)	QA de seguridad
Privacidad de las compras	CP-15	Carlos Vidal (QA)	QA de seguridad
API REST por rol	Manuales con <code>curl</code>	Carlos Vidal (QA)	QA de integracion
Integridad del stock	Manuales	Carlos Vidal (QA)	QA funcional junto a desarrollo
Revision del reporte y cierre	—	Carlos Vidal (Scrum Master)	Lider de QA

Responsabilidades del rol de QA en este proyecto

1. Traducir cada criterio de aceptacion y de rechazo en al menos un caso de prueba.
2. Mantener los *page objects* alineados con la interfaz cuando esta cambia.
3. Ejecutar la regresion completa antes de cada Pull Request y de cada promocion de rama.
4. Registrar los defectos con sus pasos de reproduccion y su severidad.
5. Custodiar las evidencias: reporte HTML y capturas de cada caso.

4.7 Plan de automatizacion de pruebas

Estrategia. Se automatiza el nivel **extremo a extremo** porque es donde se concentra el riesgo del sistema: la mayoría de los defectos posibles no estan en una funcion aislada sino en la interaccion entre el formulario, la *server action*, la comprobacion de rol y la base de datos. Cada criterio de aceptacion o de rechazo que puede observarse desde el navegador tiene su caso.

Que se automatiza y que no

Se automatiza	Se prueba a mano
Inicio de sesion, registro y sus rechazos	Cierre de sesion y caducidad de la cookie
CRUD completo de productos por la interfaz	Alta, cambio de rol y bloqueo de cuentas
Restricciones de acceso por rol en paginas	Codigos 401 y 403 de los seis endpoints de la API
Filtrado de las compras por usuario	Devolucion de stock al anular una compra
Validaciones de formulario y sus mensajes	Aspecto visual y comportamiento responsive

Arquitectura de la suite

tests-selenium/	
└─ 01-login.spec.js	HU-01 y HU-07 → CP-01, CP-02, CP-03
└─ 02-crear-producto.spec.js	HU-02 → CP-04, CP-05, CP-06
└─ 03-listar-productos.spec.js	HU-03 → CP-07, CP-08
└─ 04-editar-producto.spec.js	HU-04 → CP-09, CP-10
└─ 05-eliminar-producto.spec.js	HU-05 → CP-11
└─ 06-registro.spec.js	HU-06 → CP-12, CP-13
└─ 07-roles-y-permisos.spec.js	HU-08, HU-10 → CP-14, CP-16
└─ 08-mis-compras.spec.js	HU-09 → CP-15
└─ paginas/	Page Objects: PaginaLogin, PaginaRegistro, PaginaProductos, FormularioProducto
└─ soporte/	navegador, hooks, datos, capturas, formulario

Patrones aplicados

- **Page Object.** Cada pantalla expone metodos con nombre de negocio (`iniciarSesionValida`, `pulsarEditar`, `datosDe`). Los selectores viven en un solo lugar: si cambia el HTML, se corrige una linea y no diez casos.
- **Datos independientes.** Los nombres y correos que crea la suite llevan una marca temporal (`nombreUnico`, `correoUnico`), de modo que dos ejecuciones seguidas no colisionan.
- **Estado conocido.** El hook `beforeAll` ejecuta `npm run db:seed`, que deja siempre los mismos cuatro productos, las dos cuentas y las dos compras de ejemplo.
- **Aislamiento.** El hook `beforeEach` abre un navegador nuevo y `afterEach` lo cierra: ninguna sesion ni cookie se filtra de un caso al siguiente.
- **Evidencia automatica.** `afterEach` toma una captura pase o falle el caso y la adjunta al reporte con `addContext`.

- **Preparacion por API.** Los escenarios de edicion y borrado montan su producto mediante una llamada REST autenticada, en lugar de repetir el alta por la interfaz: lo que se automatiza por el navegador es la operacion bajo prueba, no su montaje.
- **Esperas explicitas.** No se usan pausas fijas. Se espera por condiciones (`until.urlIs`, `until.elementLocated`), lo que hace la suite mas rapida y mas estable.

Estabilidad frente a la hidratacion de React. Next entrega HTML desde el servidor y React lo hidrata mili-segundos despues. Si Selenium escribe o pulsa dentro de esa ventana, React repinta el formulario y el valor o la pulsacion se pierden **sin ningun aviso**: el caso falla por un motivo ajeno a la aplicacion. El modulo `soporte/formulario.js` resuelve ambos casos comprobando que la accion tuvo efecto y reintentando mientras no lo tenga. En la misma linea, se desactiva el gestor de contraseñas de Chrome, que tras un primer inicio de sesion autocompletaba el formulario y pisaba lo que escribia la prueba.

Ejecucion

Comando	Uso
<code>npm run test:e2e</code>	Suite completa con el navegador visible, para depurar
<code>npm run test:e2e:headless</code>	Suite completa sin interfaz grafica, para la regresion
<code>npx mocha tests-selenium/07-roles-y-permisos.spec.js</code>	Un unico archivo de casos

Mantenimiento. Cuando una historia cambia, primero se actualizan sus criterios en `docs/historias-usuario.md`, despues el caso de prueba y por ultimo la implementacion. Si un caso falla de forma intermitente se trata como un defecto de la suite y se corrige antes de seguir: una prueba en la que no se confia es peor que no tenerla.

4.8 Ejecucion y demostracion

Resultado de la regresion final

Metrica	Valor
Suites ejecutadas	8
Casos ejecutados	16
Superados	16
Fallidos	0
Duracion	~29 s en modo headless
Fecha	14 de agosto de 2026

Cobertura por tipo de prueba

Tipo	Casos	Total
Camino feliz	CP-01, CP-04, CP-07, CP-09, CP-11, CP-12, CP-15	7
Negativa	CP-02, CP-05, CP-08, CP-10, CP-13, CP-14, CP-16	7
Limites	CP-03, CP-06	2

Evidencias

- **Reporte HTML:** `tests-selenium/reportes/reporte.html`, generado por mochawesome, autocontenido y con los tiempos de cada caso.
- **Reporte JSON:** `tests-selenium/reportes/reporte.json`, con las estadísticas de la corrida.
- **Capturas automaticas:** `tests-selenium/reportes/capturas/`, una por caso, tomadas pase o falle y enlazadas desde el reporte.
- **Video demostrativo:** enlace en la [seccion 5](#).

Como reproducir la ejecucion

```
git clone https://github.com/cvidalmelo/rd-market.git
cd rd-market
npm install
cp .env.example .env          # ajusta BETTER_AUTH_SECRET
npm run db:migrate
npm run db:seed

npm run dev                    # terminal 1: deja la app en http://localhost:3000
npm run test:e2e:headless      # terminal 2: ejecuta los 16 casos
open tests-selenium/reportes/reporte.html
```

Pruebas manuales de la API por rol

```
# Sin sesion -> 401
curl -i http://localhost:3000/api/productos

# Como cliente -> 200 al leer, 403 al escribir
curl -c cliente.txt -X POST http://localhost:3000/api/auth/sign-in/email \
  -H 'Content-Type: application/json' -H 'Origin: http://localhost:3000' \
  -d '{"email":"carlos@minimarket.com","password":"carlos1234"}'
curl -i -b cliente.txt http://localhost:3000/api/productos
curl -i -b cliente.txt -X POST http://localhost:3000/api/productos \
  -H 'Content-Type: application/json' -d '{"nombre":"X","precio":1,"stock":1}'

# Como administradora -> 201
curl -c admin.txt -X POST http://localhost:3000/api/auth/sign-in/email \
  -H 'Content-Type: application/json' -H 'Origin: http://localhost:3000' \
  -d '{"email":"ana@minimarket.com","password":"ana1234"}'
curl -i -b admin.txt -X POST http://localhost:3000/api/productos \
  -H 'Content-Type: application/json' -d '{"nombre":"Prueba","precio":1,"stock":1}'
```

Resultado obtenido: **401** sin sesion, **200/403** con rol cliente y **201** con rol administrador, en linea con RF-12 y RNF-04.

5. Demostracion y entregables

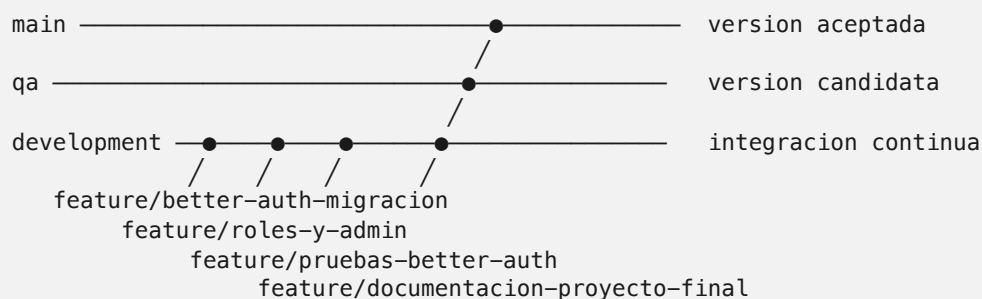
Entregable	Enlace
Repositorio de codigo	https://github.com/cvidalmelo/rd-market
Tablero de historias (Jira)	https://carlosvidalmelo.atlassian.net/jira/software/c/projects/MM4/issues
Codigo de las pruebas automatizadas	https://github.com/cvidalmelo/rd-market/tree/main/tests-selenium
Reporte HTML de pruebas	tests-selenium/reportes/reporte.html en el repositorio
Video demostrativo	[PENDIENTE DE ENLACE]

Contenido del video. El video muestra el incremento del primer Release: registro de una cuenta nueva, acceso como cliente comprobando que no ve la administracion ni puede modificar el catalogo, acceso como administradora gestionando usuarios y productos, la diferencia entre "Mis compras" y el listado completo, y la ejecucion en vivo de los 16 casos automatizados con el reporte HTML resultante.

Estructura del repositorio

```
rd-market/  
├── docs/           Historias de usuario y este documento  
├── prisma/         Esquema, migraciones y datos de ejemplo  
├── src/  
│   ├── app/       Paginas, server actions y API REST  
│   ├── components/ Componentes e interfaz compartida  
│   ├── lib/        Better Auth, acceso a datos y reglas de negocio  
│   └── proxy.ts    Proteccion optimista de rutas  
└── tests-selenium/ Suite de pruebas, page objects y reportes
```

Flujo de ramas. El historial refleja el proceso completo: cada funcionalidad entra por una rama `feature/*` y un Pull Request, `development` recoge el trabajo del sprint, `qa` recibe la version candidata y `main` la version aceptada.



6. Conclusiones

Sobre el producto. MiniMarket cumple el objetivo planteado: es un sistema funcional de gestion de inventario y ventas en el que cada persona ve y hace unicamente lo que su rol permite. El salto de calidad de esta entrega no esta en anadir pantallas, sino en que el acceso dejo de ser un adorno. En la version anterior las contraseñas se guardaban en texto plano, se comparaban con un `!=="` y la API REST estaba completamente abierta; ahora las contraseñas se cifran con `scrypt`, las sesiones viven en la base de datos y se pueden revocar, y los seis endpoints comprueban sesion y rol.

Sobre la decision de usar Better Auth. Escribir autenticacion propia es una de las formas mas seguras de introducir un fallo de seguridad. Delegar en una libreria especializada elimino de golpe el hasheo manual, la firma de tokens y la gestion del ciclo de vida de la sesion, y su plugin de administracion aporoto roles, bloqueo de cuentas y suplantacion sin escribir esa logica. El coste fue migrar el modelo de datos: el modelo `Usuario` propio se sustituyo por el `User` de la libreria y las compras se repuntaron a la nueva tabla. Fue la decision correcta: el codigo resultante es mas corto y hace mas.

Sobre la autorizacion. La leccion mas util del sprint 4 fue que **ocultar un boton no es proteger una funcion**. La primera version escondia el enlace de usuarios a la clientela, pero la ruta seguia respondiendo si se escribia a mano y la API nunca preguntaba quien llamaba. El diseno definitivo comprueba el rol en tres capas —pagina, *server action* y route handler— y son precisamente CP-14 y CP-16 los casos que verifican que el atajo por la barra de direcciones no funciona.

Sobre las pruebas automatizadas. Los 16 casos cubren las diez historias y detectaron problemas reales durante el desarrollo. El aprendizaje mas valioso fue distinguir un defecto de la aplicacion de un defecto de la suite: dos casos fallaban de forma intermitente y la causa no estaba en el codigo del sistema, sino en que Selenium interactuaba con el formulario antes de que React lo hidratara, y en el gestor de contraseñas de Chrome autocompletando el login. Diagnosticarlo llevo mas tiempo que escribir los casos nuevos, y confirmo que una prueba inestable erosiona la confianza en toda la bateria.

Sobre la metodologia. Trabajar solo con Scrum obliga a ser deliberado. La separacion de roles no es un tramite: redactar la historia como Product Owner antes de programarla como desarrollador fuerza a decidir que significa "terminado" **antes** de escribir la primera linea, y los criterios de rechazo se convierten casi mecanicamente en pruebas negativas. El flujo de ramas con Pull Requests cumple la funcion de la revision por pares: al describir el cambio para el PR, mas de una vez aparecio algo que corregir.

Trabajo futuro. Las lineas naturales de continuacion son la verificacion de correo y la recuperacion de contraseña —Better Auth ya las soporta—, el acceso con proveedores externos, la migracion de SQLite a PostgreSQL para un despliegue real, y anadir pruebas de integracion sobre la API que hoy se comprueba a mano.

7. Bibliografia

1. Better Auth. (2026). *Better Auth Documentation*. Recuperado de <https://better-auth.com/docs>
2. Better Auth. (2026). *Admin plugin*. Recuperado de <https://better-auth.com/docs/plugins/admin>
3. Better Auth. (2026). *Prisma adapter*. Recuperado de <https://better-auth.com/docs/adapters/prisma>
4. Better Auth. (2026). *Next.js integration*. Recuperado de <https://better-auth.com/docs/integrations/next>
5. Vercel. (2026). *Next.js Documentation — App Router*. Recuperado de <https://nextjs.org/docs>
6. Vercel. (2026). *Authentication*. Recuperado de <https://nextjs.org/docs/app/guides/authentication>
7. Prisma. (2026). *Prisma ORM Documentation*. Recuperado de <https://www.prisma.io/docs>
8. SeleniumHQ. (2026). *Selenium WebDriver Documentation*. Recuperado de <https://www.selenium.dev/documentation/webdriver/>
9. Mocha. (2026). *Mocha Documentation*. Recuperado de <https://mochajs.org/>
10. Chai. (2026). *Chai Assertion Library*. Recuperado de <https://www.chaijs.com/>
11. mochawesome. (2026). *mochawesome — reporter for Mocha*. Recuperado de <https://github.com/adamgruber/mochawesome>
12. Schwaber, K., y Sutherland, J. (2020). *La Guia de Scrum*. Recuperado de <https://scrumguides.org/>
13. Cohn, M. (2004). *User Stories Applied: For Agile Software Development*. Addison-Wesley.
14. Atlassian. (2026). *Jira Software Documentation*. Recuperado de <https://support.atlassian.com/jira-software-cloud/>
15. OWASP Foundation. (2026). *Authentication Cheat Sheet*. Recuperado de https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
16. Fowler, M. (2013). *PageObject*. Recuperado de <https://martinfowler.com/bliki/PageObject.html>